

## MODULE 7

# Build a Model and Deploy It

**Three models, Fan data, scoreboard comparison**

TinyML Course · Day 3

## Warm-up — three questions from Day 2

1. Which fan state is nearly **invisible to RMS** alone — and which feature family exposes it?
2. Trees or neural nets — which one needs **feature scaling**, and why?
3. What does `emlearn.convert(model).save()` actually produce?

*Two minutes with your neighbour — then we compare notes.*

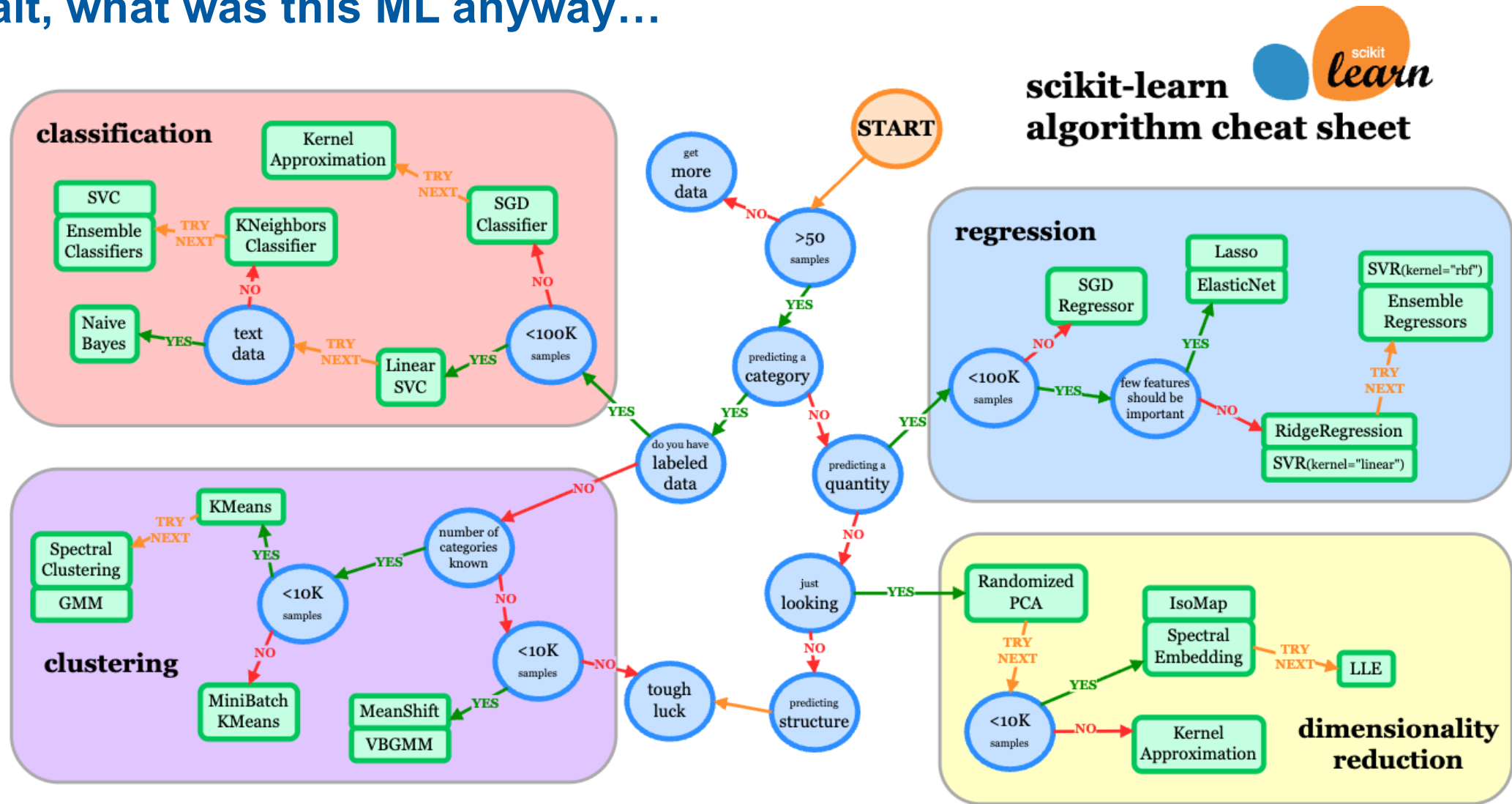
## Where we are in the pipeline

sensor → acquisition → features → MODEL → deployment → decision  
(M2) (M3–M4) (M5) (TODAY) (TODAY) (M8)

- Dataset: your PC-fan recordings — `normal` , `imbalance` , `blocked` , `scrape` , `off`
- Features: the 13-feature **time-domain baseline** — std / MAD / kurtosis / RMS per axis + resultant (Module 5)
- Frameworks: emlearn + Edge Impulse (Module 6)

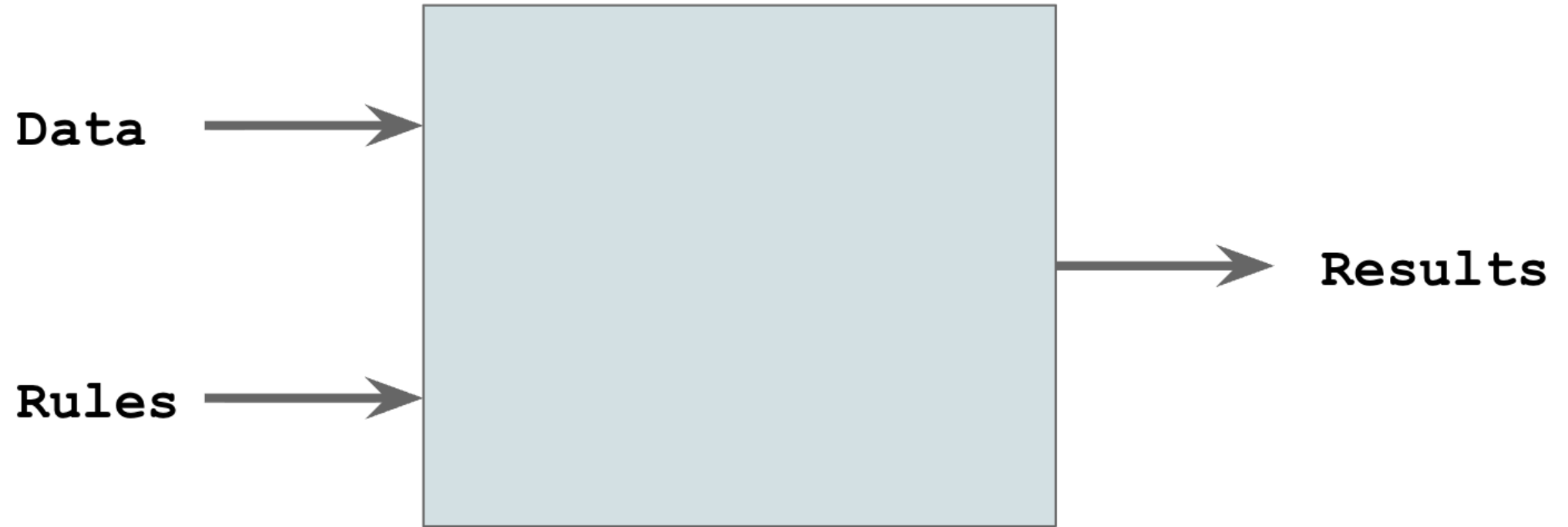
**Today:** train three model families on the *same* data, deploy all of them to the RAK4631, and measure.

# Wait, what was this ML anyway...

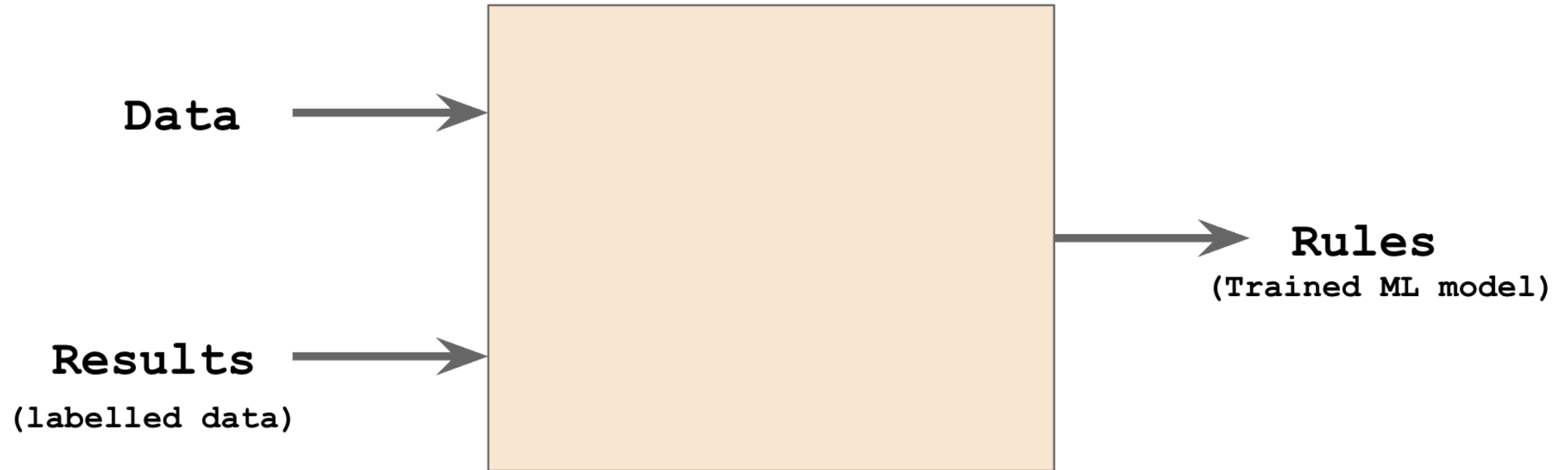


[https://scikit-learn.org/stable/machine\\_learning\\_map.html](https://scikit-learn.org/stable/machine_learning_map.html)

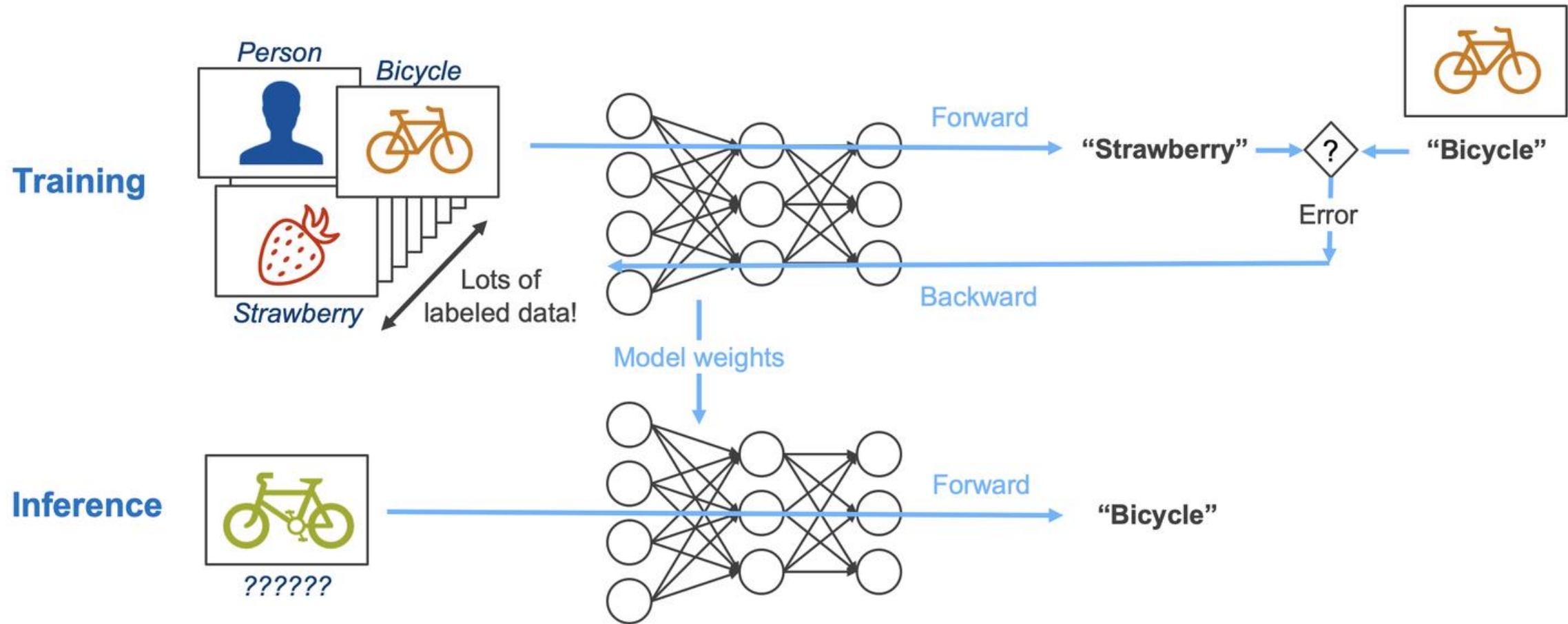
# Rule based / traditional programming



# Data driven / ML programming

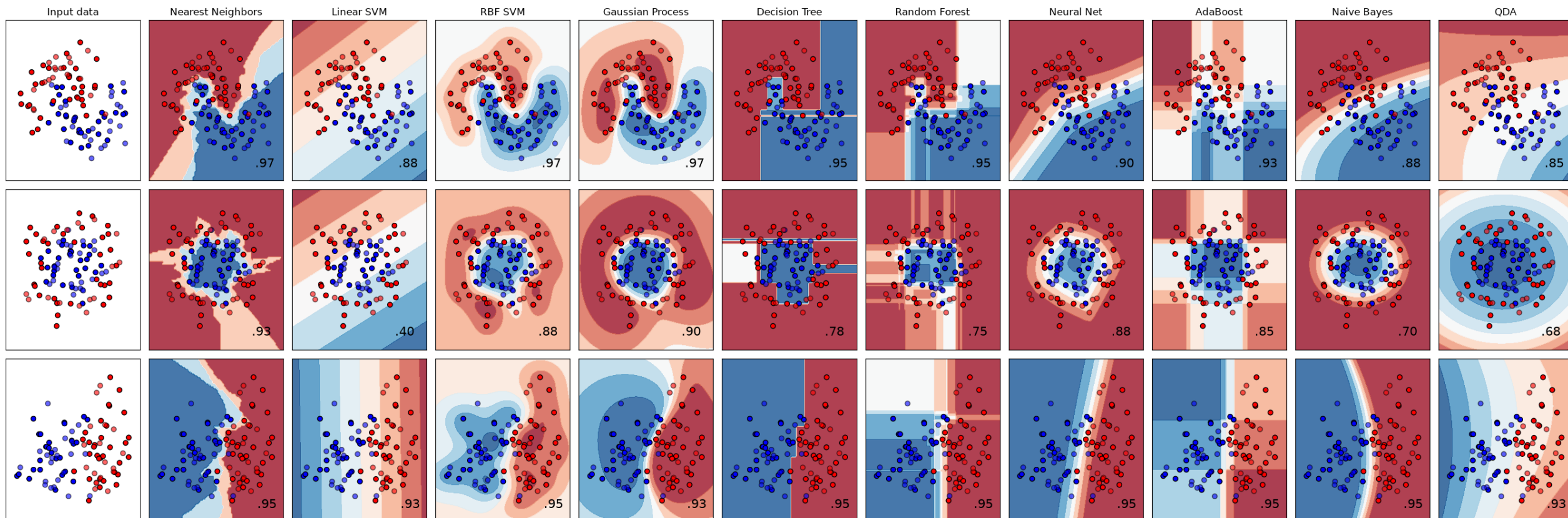


# Train vs Inference



# Deep learning is not always the answer

## Classifiers & decision boundaries



[https://scikit-learn.org/stable/auto\\_examples/classification/plot\\_classifier\\_comparison.html](https://scikit-learn.org/stable/auto_examples/classification/plot_classifier_comparison.html)

## The contenders

|          | Model         | Input                   | Toolchain              | Deploys as  |
|----------|---------------|-------------------------|------------------------|-------------|
| <b>A</b> | Random Forest | 13 handcrafted features | scikit-learn + emlearn | C header    |
| <b>B</b> | Dense NN      | DSP-block features      | Edge Impulse Studio    | C++ library |
| <b>C</b> | 1D-CNN        | raw 3-axis windows      | Edge Impulse Studio    | C++ library |

Same fan. Same labels. Same MCU (nRF52840: 256 KB RAM, 1 MB flash, 64 MHz M4F).

**Question of the day: which one earns its place in a product?**

## Recap — the fan dataset

- 3-axis vibration from the RAK1904 (LIS3DH, sensor slot A, I<sup>2</sup>C `0x18` )
- Sampled at **250 Hz**, windowed into **500-sample windows** (2 s), 50 % overlap
- Five machine states = five class labels
- Train/test split done **by recording session**, not by window — no leakage

Features from windows in training **must** equal windows in firmware.

## Model A — Random Forest

- A **decision tree** = learned cascade of `if (feature < threshold)` splits
- A **forest** = many trees, each trained on a random data/feature subset → majority vote
- Handles non-linear boundaries, ignores irrelevant features, hard to overfit with enough trees

```
if (rms_x < 0.13)    → off  
else if (std_y < 0.02) → normal  
else if (kurt_z > 3.1) → scrape  
... × 10 trees → vote
```

A random forest **is** embedded-friendly control flow.

## Why trees shine on microcontrollers

- **No feature scaling required** — splits compare raw values; skip the StandardScaler entirely
- **No multiplications** — inference is comparisons + branches; runs in integer maths
- **Tiny and predictable** — footprint  $\approx$  number of decision nodes; worst-case latency  $\approx$  tree depth
- emlearn: portable **header-only C99**, zero dependencies

"Tree ensembles are competitive with deep learning on feature data at 10–100× fewer resources"  
— J. Nordby, *Practical TinyML with emlearn* (Module 6 deck)

## Model A's feature set — spot the gap

The 13-feature baseline: std · MAD · kurtosis · RMS per axis, + resultant.

**All time-domain. Not one spectral feature.**

- Module 5 proved amplitude statistics are *frequency-blind*
- **Predict now, before training: which class will suffer in the confusion matrix?**
- Write your prediction on the worksheet — today's shoot-out will grade it

## Train → convert → header

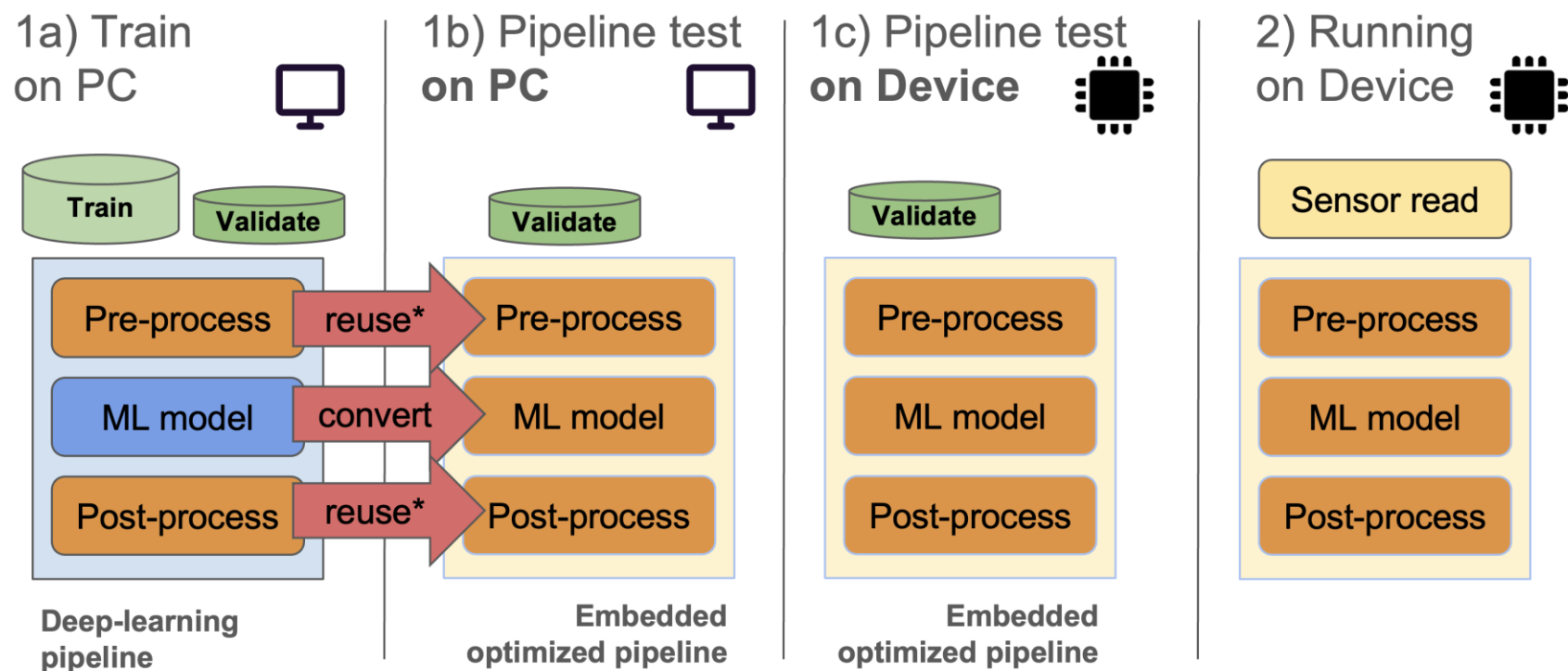
```
from sklearn.ensemble import RandomForestClassifier
import emlearn

clf = RandomForestClassifier(n_estimators=10, max_depth=8)
clf.fit(X_train, y_train)      # X = 13 fan features
print(clf.score(X_test, y_test))

cmodel = emlearn.convert(clf, method='inline') # dtype: 'float' for now
cmodel.save(file='fan_model.h', name='fan_model')
```

- `fan_model.h` is plain C — `#include` it, done
- `dtype='int16_t'` etc. exists → we quantise trees in **Module 9**

## Deploy in steps —4-step ladder



1. Python features + Python model ✓ (notebook)
2. **C features + C model on your laptop** — same static window → same class?
3. C features + C model on the RAK4631, static test arrays
4. Live sensor data

## Golden test vectors

```
// one 200-sample window of a known 'imbalance' recording, from CSV
static const float test_imbalance[200][3] = {
    { 0.012f, -0.981f, 0.033f },
    { 0.018f, -0.976f, 0.041f },
    /* ... exported by train_rf.py ... */
};
```

- Exported automatically by the training script
- Expected: `fan_model_predict(...) == IMBALANCE` **before** you trust live data
- Python and C features must agree to ~4 decimals (Module 5 method)

```
const float samples[3][150] = {
    {
        0.250000, 0.250000, 0.260000, 0.250000, 0.250000, 0.250000, 0.260000, 0.270000, 0.270000, 0.270000, 0.260000, 0.260000,
    },
    {
        -0.500000, -0.400000, -0.450000, -0.450000, -0.470000, -0.470000, -0.470000, -0.480000, -0.490000, -0.490000, -0.480000,
    },
    {
        -0.960000, -0.860000, -0.840000, -0.830000, -0.830000, -0.820000, -0.830000, -0.830000, -0.840000, -0.840000, -0.840000,
    }
};
```



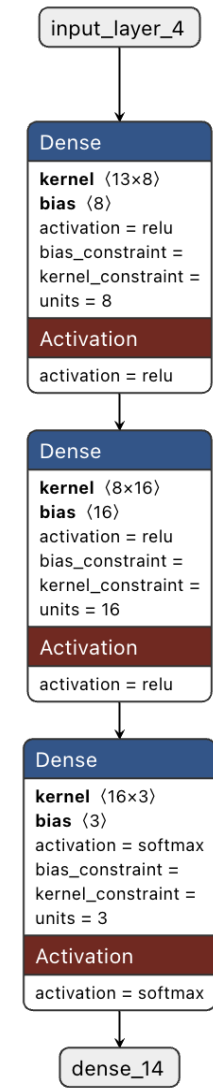
## Start lab track A now — RF training runs while we continue

`exercises/module7-models/rf-features/`

- `ei_json_to_csv.py` → `train_rf.py` → `fan_model.h` (a few minutes end-to-end)
- Leave the PlatformIO build for the lab block — just get the header generated
- **Done when:** `fan_model.h` exists and reports its size

## Model B — Dense NN on features

- Input: 13 features → **Dense 8 (ReLU)** → **Dense 16 (ReLU)** → **Dense 5 (softmax)**
- ~1 k parameters ≈ 4 KB as float32 — trivial for 1 MB flash
- Learns feature *combinations* the tree splits can't express smoothly
- In the lab: built visually in **Edge Impulse Studio** (Spectral Analysis → NN Classifier)



## NNs need scaled inputs (trees don't)

```
scaler = StandardScaler().fit(X_train)    #  $z = (x - \text{mean}) / \text{std}$ 
```

```
// exported from the notebook — apply before every inference
static const float offset[13] = { 0.0928f, 0.1283f, /* ... */ };
static const float scale[13] = { 0.0952f, 0.1067f, /* ... */ };
for (int i = 0; i < 13; i++)
    features[i] = (features[i] - offset[i]) / scale[i];
```

- Forget this → the model happily predicts garbage (softmax always sums to 1!)
- Edge Impulse handles it inside the DSP block — one reason SaaS is comfortable

## Edge Impulse impulse design (Model B)

|                  |                     |                     |
|------------------|---------------------|---------------------|
| Time series data | Spectral Analysis   | NN Classifier       |
| window 2000 ms   | → (per-axis filter, | → Dense 20 → 10 → 5 |
| increase 500 ms  | FFT, PSD features)  | softmax             |

- **Spectral Analysis** = your Module 5 features, industrialised (RMS + FFT band powers — fan RPM harmonics!)
- Keep the window equal to your recorded window
- Train, inspect the confusion matrix, then **Live classification** before deploying



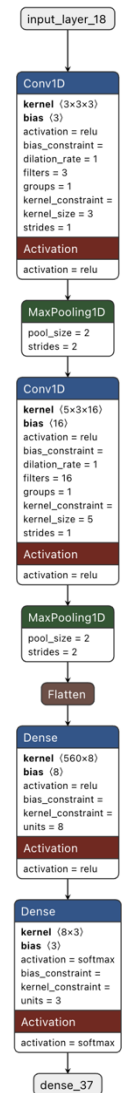
## Queue lab track B now — EI trains server-side

In your EI fan project:

- Impulse: Spectral Analysis → NN classifier (defaults are fine to start)
- Hit **train** — results wait for you; compare val. accuracy to track A later
- **Done when:** the training job is running

## Model C — 1D-CNN on raw windows

- Input: raw **200 × 3** window — no handcrafted features
- Conv1D(3, k=3) → MaxPool → Conv1D(6, k=5) → MaxPool → Flatten → Dense 8 → softmax
- Kernels = **learned** feature extractors (period detectors, spike detectors...)
- Old-course grid search: small models won — 3 filters/16 units ≈ 80 %, 8 filters/64 units *worse* (overfit)



## Handcrafted features vs raw data

|                    | Features + RF/NN                       | CNN on raw                           |
|--------------------|----------------------------------------|--------------------------------------|
| Domain knowledge   | encoded by <i>you</i> (RPM, harmonics) | must be <i>learned</i> from data     |
| Data appetite      | small datasets OK                      | wants much more data                 |
| MCU cost           | feature calc + tiny model              | bigger model, more RAM (activations) |
| Explainability     | feature importances                    | low                                  |
| New sensor/machine | re-engineer features                   | often just retrain                   |

**Fan verdict (spoiler):** with 20 min of data per class, features usually win. Verify it yourself.

## Deploying the EI models into PlatformIO

1. Studio → **Deployment** → *Arduino library* → Build → download zip
2. Unzip into your project's `lib/` folder:

```
fan-monitor/  
├── lib/  
│   ├── Fan_Monitor_inferencing/ ← unzipped EI export, correct project name!  
│   │   └── src/ (edge-impulse-sdk/, model-parameters/, tflite-model/)  
├── src/main.cpp  
└── platformio.ini
```

3. `#include <Fan_Monitor_inferencing.h>` — PlatformIO compiles the SDK automatically

## run\_classifier()

in 15 lines

```
static float window[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE]; // 200 × 3, interleaved

signal_t signal;
numpy::signal_from_buffer(window,
    EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE, &signal);

ei_impulse_result_t result;
uint32_t t0 = micros();
EI_IMPULSE_ERROR err = run_classifier(&signal, &result, false);
uint32_t dt = micros() - t0; // ← latency, for the scoreboard

for (size_t i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++)
    Serial.printf("%s: %.3f\n", result.classification[i].label,
        result.classification[i].value);
```

# The canonical on-device ML loop

Every embedded ML application has the same skeleton — here as keyword spotting:

## Keyword Spotting Components

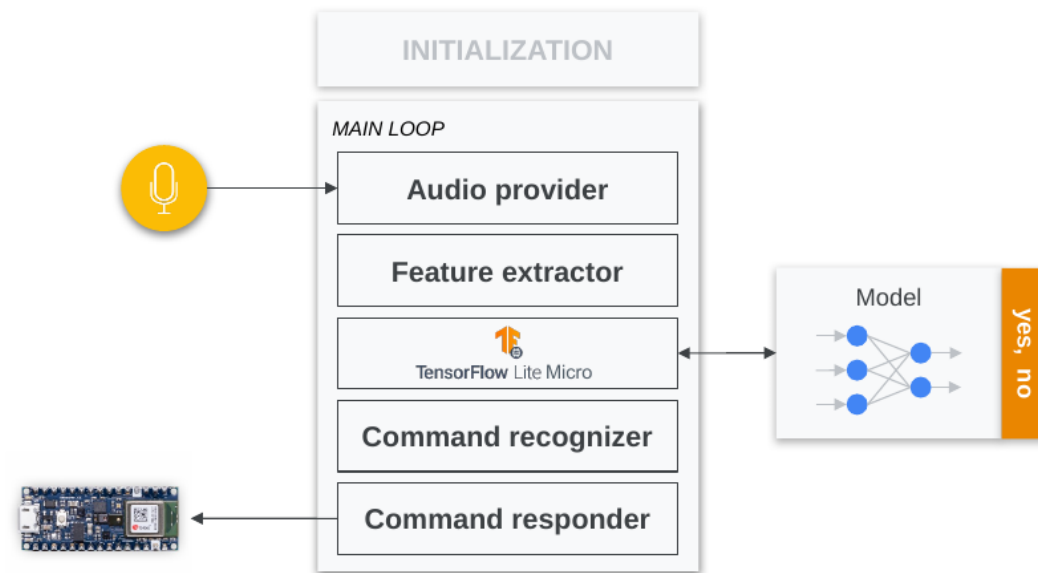
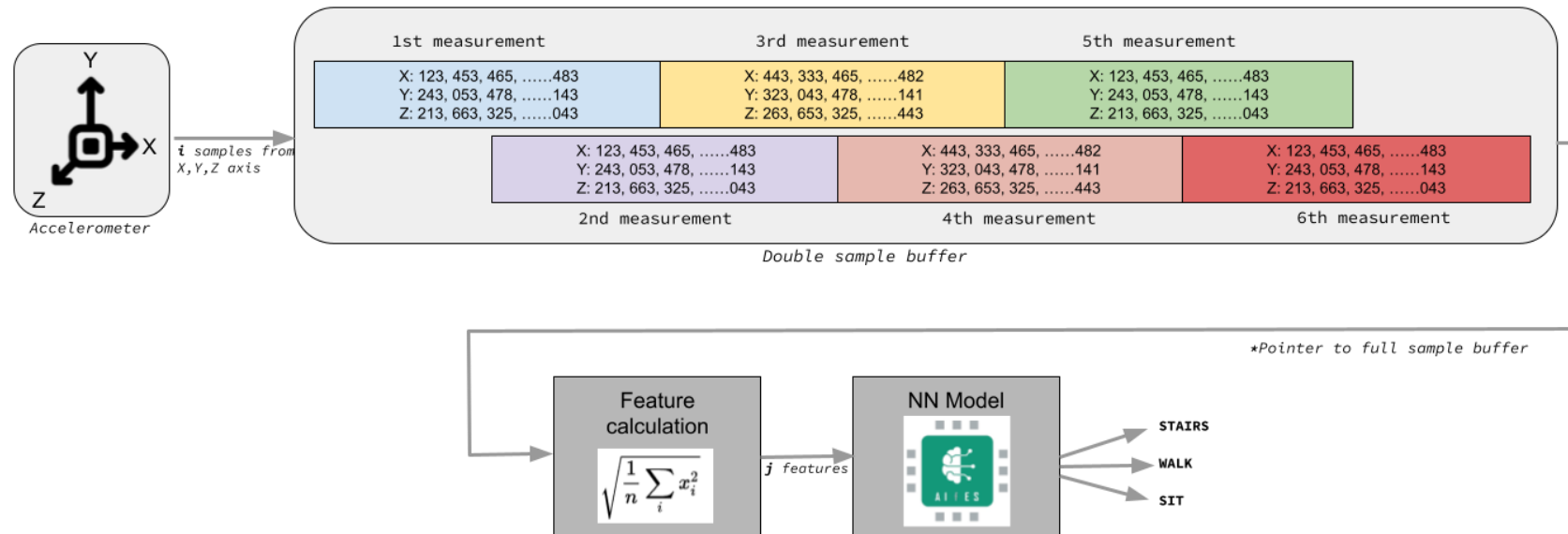


Figure: TinyML on edX (HarvardX), V.J. Reddi et al. — used with permission

# Firmware architecture — double buffering



- Fill buffer B while computing features + inference on buffer A, then swap
- 50 % overlap → a prediction every second instead of every two
- A tiny state machine ( **SAMPLING → INFER → REPORT** ) keeps it simple

## Measuring the contenders

| Metric       | How                                                                                     |
|--------------|-----------------------------------------------------------------------------------------|
| Accuracy     | held-out test set (Studio <i>Model testing</i> / notebook confusion matrix)             |
| Flash        | PlatformIO build summary — model build minus empty baseline build                       |
| RAM (static) | build summary + <code>EI_CLASSIFIER_*</code> estimates in <code>model_metadata.h</code> |
| Latency      | <code>micros()</code> around inference, median of $\geq 20$ runs                        |

RAM: [==== ] 38.2% (used 100 kB from 256 kB)

Flash: [=== ] 31.0% (used 316 kB from 1 MB) ← write these down

## The scoreboard

| Model                  | Accuracy | Flash $\Delta$ | RAM $\Delta$ | Latency       |
|------------------------|----------|----------------|--------------|---------------|
| RF (emlearn, float)    | _____ %  | _____ kB       | _____ kB     | _____ $\mu$ s |
| Dense NN (EI, float32) | _____ %  | _____ kB       | _____ kB     | _____ ms      |
| 1D-CNN (EI, float32)   | _____ %  | _____ kB       | _____ kB     | _____ ms      |

Typical order of magnitude: RF tens of  $\mu$ s / a few kB · dense NN ~ms · CNN several ms + tens of kB RAM.

**Fill it in during the lab — this worksheet is the module deliverable.**

## The reveal — and the fix

- Baseline RF confusion matrix: `blocked` ↔ `normal` — was that your prediction?
- Why: `blocked` 's signature is an **RPM shift** — a frequency feature; time-domain stats never stood a chance
- **The fix you already own:** append the 5 **band energies** from your Module 5 feature code (one axis first: 13 → 18 features) → retrain → convert → redeploy
- Watch `blocked` separate — *documented extension in exercise 7.1*

Same forest, same data — better **features**.

## What usually wins, and why

- **Features + RF**: best accuracy-per-byte on small tabular/vibration datasets; start here
- **Features + small NN**: worth it when class boundaries are smooth/combinatorial; needs scaling
- **CNN on raw**: pays off with *lots* of data, or when features are unknown (audio!)
- All three share a blind spot: they **only know the five states we taught them**

A worn bearing you never recorded will be confidently classified as... something. Module 8 fixes this.

## Exercise hand-off — Module 7 lab

[exercises/module7-models/](#)

1. **7.1 — RF via emlearn** ( [rf-features/](#) ): train on your fan features, convert, deploy with live LIS3DH sampling, LED + serial output (*mandatory*) — **extension:** add the Module 5 band energies (13 → 18 features), retrain, watch [blocked](#) separate
  2. **7.2 — EI deploy** ( [ei-deploy/](#) ): dense-NN impulse → Arduino-library export → PlatformIO, [run\\_classifier](#) on the live window
  3. **7.3 — CNN track:** second EI impulse on raw data; redeploy; compare
  4. **Worksheet:** fill the scoreboard ( [comparison-worksheet.md](#) ) — accuracy / flash / RAM / latency per model
- Checkpoint (end of block):** live fan classification on serial + a completed scoreboard row for  $\geq 2$  models.

## Checkpoint & bridge

- Compare scoreboards across teams — why do identical models score differently? (*hint: it's the data*)
- Keep the EI project open: Module 8 adds an **anomaly block** next to your classifier
- Keep the PlatformIO project: Module 9 will re-deploy it **quantised**

**Next: Module 8 — what happens when the fan breaks in a way you never recorded?**